

# 【学生方阵】

## 题目描述

学校组织活动，将学生排成一个矩形方阵。

请在矩形方阵中找到最大的位置相连的男生数量。这个相连位置在一个直线上，方向可以是水平的，垂直的，成对角线的或者呈反对角线的。

注：学生个数不会超过10000

## 输入描述

输入的第一行为矩阵的行数和列数，

接下来的 n 行为矩阵元素，元素间用 "," 分隔。

## 输出描述

输出一个整数，表示矩阵中最长的位置相连的男生个数。

## 用例

|    |                                      |
|----|--------------------------------------|
| 输入 | 3,4<br>F,M,M,F<br>F,M,M,F<br>F,F,F,M |
| 输出 | 3                                    |

|    |  |   |   |   |   |
|----|--|---|---|---|---|
| 说明 |  |   |   |   |   |
|    |  | F | M | M | F |
|    |  | F | M | M | F |
|    |  | F | F | F | M |

## 题目解析

本题的解题思路其实不难，遍历查找矩阵中每一个M点，然后求该M点的水平、垂直、正对角线、反对角线，四个方向的M点个数，然后保留最大的个数，就是题解。

但是这种方法会存在很多 **重复的** 查找，比如

|  |   |   |   |   |
|--|---|---|---|---|
|  |   |   |   |   |
|  | F | M | M | F |
|  | F | M | M | F |
|  | F | F | F | M |
|  |   |   |   |   |
|  | F | M | M | F |
|  | F | M | M | F |
|  | F | F | F | M |

红色M是当前遍历到的M，绿色M是以红色M为原点查找到的M，如上图两个红色M点会重复查找同一条M链。

为了避免这种重复查找，我们可以增加判断：

如果当前M点的

- 左上角点是M，则反对角线不用查找了
- 右上角点是M，则正对角线不用查找了
- 上边点是M，则垂直线不用查找了
- 左边点是M，最水平线不用查找了

|  |   |   |   |   |
|--|---|---|---|---|
|  | F | M | M | F |
|  | F | M | M | F |
|  | F | F | F | M |

CSDN @伏城之外

如上图红色M的左上、上、左点都是M，因此红色M的

- 反对角线已经被其左上点M查找过了，因此不用找了，
- 垂直线已经被其上边点M查找过了，因此不用找了
- 水平线已经被其左边点M查找过了，因此不用找了

## Java算法源码

```
1 import java.util.Scanner;
2
3 public class Main {
4     static int n;
5     static int m;
6     static String[][] matrix;
7
8     public static void main(String[] args) {
9         Scanner sc = new Scanner(System.in).useDelimiter("[,\\n]");
```

```

10 | 11 |     n = sc.nextInt();
12 |     m = sc.nextInt();
13 |
14 |     matrix = new String[n][m];
15 |     for (int i = 0; i < n; i++) {
16 |         for (int j = 0; j < m; j++) {
17 |             matrix[i][j] = sc.next();
18 |         }
19 |     }
20 |
21 |     System.out.println(getResult());
22 | }
23 |
24 | public static int getResult() {
25 |     int ans = 0;
26 |
27 |     int[][] offsets = {{0, 1}, {1, 0}, {1, 1}, {1, -1}};
28 |
29 |     for (int i = 0; i < n; i++) {
30 |         for (int j = 0; j < m; j++) {
31 |             if ("M".equals(matrix[i][j])) {
32 |                 for (int[] offset : offsets) {
33 |                     int oldI = i - offset[0];
34 |                     int oldJ = j - offset[1];
35 |
36 |                     if (oldI >= 0 && oldI < n && oldJ >= 0 && oldJ < m && "M".equals(matrix[oldI][oldJ])) {
37 |                         continue;
38 |                     }
39 |
40 |                     int len = 1;
41 |                     int newI = i + offset[0];
42 |                     int newJ = j + offset[1];
43 |
44 |                     while (newI >= 0
45 |                         && newI < n
46 |                         && newJ >= 0
47 |                         && newJ < m

```

```

48         && "M".equals(matrix[newI][newJ])) {
49             len++;
50             newI += offset[0];
51             newJ += offset[1];
52         }
53
54         ans = Math.max(ans, len);
55     }
56 }
57 }
58 }
59
60 return ans;
61 }
62 }

```

## JS算法源码

```

1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const [n, m] = (await readline()).split(",").map(Number);
7
8      const matrix = [];
9      for (let i = 0; i < n; i++) {
10         matrix.push((await readline()).split(","));
11     }
12
13     console.log(getResult(matrix, n, m));
14 })();
15
16 function getResult(matrix, n, m) {
17     let ans = 0;

```

```
18 |
19 |     const offsets = [
20 |       [0, 1],
21 |       [1, 0],
22 |       [1, 1],
23 |       [1, -1],
24 |     ];
25 |
26 |     for (let i = 0; i < n; i++) {
27 |       for (let j = 0; j < m; j++) {
28 |         if (matrix[i][j] == "M") {
29 |           for (let offset of offsets) {
30 |             const oldI = i - offset[0];
31 |             const oldJ = j - offset[1];
32 |
33 |             if (
34 |               oldI >= 0 &&
35 |               oldI < n &&
36 |               oldJ >= 0 &&
37 |               oldJ < m &&
38 |               matrix[oldI][oldJ] == "M"
39 |             ) {
40 |               continue;
41 |             }
42 |
43 |             let len = 1;
44 |             let newI = i + offset[0];
45 |             let newJ = j + offset[1];
46 |
47 |             while (
48 |               newI >= 0 &&
49 |               newI < n &&
50 |               newJ >= 0 &&
51 |               newJ < m &&
52 |               matrix[newI][newJ] == "M"
53 |             ) {
54 |               len++;
```

```

55         newI += offset[0];56         newJ += offset[1];
57     }
58
59     ans = Math.max(ans, len);
60 }
61 }
62 }
63 }
64
65 return ans;
66 }

```

## Python算法源码

```

1  # 输入获取
2  n, m = map(int, input().split(","))
3  matrix = [input().split(",") for _ in range(n)]
4
5
6  # 算法入口
7  def getResult():
8      ans = 0
9
10     offsets = ((0, 1), (1, 0), (1, 1), (1, -1))
11
12     for i in range(n):
13         for j in range(m):
14             if matrix[i][j] == "M":
15                 for offset in offsets:
16                     oldI = i - offset[0]
17                     oldJ = j - offset[1]
18
19                     if n > oldI >= 0 and m > oldJ >= 0 and matrix[oldI][oldJ] == "M":
20                         continue

```

```

21 | 22 |         length = 1
23 |         newI = i + offset[0]
24 |         newJ = j + offset[1]
25 |
26 |         while n > newI >= 0 and m > newJ >= 0 and matrix[newI][newJ] == "M":
27 |             length += 1
28 |             newI += offset[0]
29 |             newJ += offset[1]
30 |
31 |         ans = max(ans, length)
32 |
33 |     return ans
34 |
35 |
36 | # 调用算法
37 | print(getResult())

```

## C算法源码

```

1 | #include <stdio.h>
2 | #include <math.h>
3 |
4 | int main() {
5 |     int n, m;
6 |     scanf("%d,%d\n", &n, &m);
7 |
8 |     char matrix[n][m];
9 |     for (int i = 0; i < n; i++) {
10 |         for (int j = 0; j < m; j++) {
11 |             scanf("%c", &matrix[i][j]);
12 |             getchar();
13 |         }
14 |     }
15 |

```

运行



```

16     int ans = 0;
17
18     int offsets[4][2] = {{0, 1},
19                          {1, 0},
20                          {1, 1},
21                          {1, -1}};
22
23     for (int i = 0; i < n; i++) {
24         for (int j = 0; j < m; j++) {
25             if (matrix[i][j] == 'M') {
26                 for (int k = 0; k < 4; k++) {
27                     int offsetI = offsets[k][0];
28                     int offsetJ = offsets[k][1];
29
30                     int oldI = i - offsetI;
31                     int oldJ = j - offsetJ;
32
33                     if (oldI >= 0 && oldI < n && oldJ >= 0 && oldJ < m && matrix[oldI][oldJ] == 'M') {
34                         continue;
35                     }
36
37                     int len = 1;
38                     int newI = i + offsetI;
39                     int newJ = j + offsetJ;
40
41                     while (newI >= 0 && newI < n && newJ >= 0 && newJ < m && matrix[newI][newJ] == 'M') {
42                         len++;
43                         newI += offsetI;
44                         newJ += offsetJ;
45                     }
46
47                     ans = (int) fmax(ans, len);
48                 }
49             }
50         }
51     }
52

```

```
53 |     printf("%d\n", ans);
    |                                     54 |
55 |     return 0;
56 | }
```

## C++ 算法源码

```
1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int main() {
6 |     int n;
7 |     cin >> n;
8 |
9 |     getchar(); // 读取逗号
10 |
11 |     int m;
12 |     cin >> m;
13 |
14 |     vector<vector<char>> matrix(n, vector<char>(m));
15 |     for (int i = 0; i < n; i++) {
16 |         for (int j = 0; j < m; j++) {
17 |             cin >> matrix[i][j];
18 |             getchar(); // 读取逗号或者换行
19 |         }
20 |     }
21 |
22 |     int ans = 0;
23 |
24 |     int offsets[4][2] = {{0, 1},
25 |                          {1, 0},
26 |                          {1, 1},
27 |                          {1, -1}};
```

```

28 |
29 |     for (int i = 0; i < n; i++) {
30 |         for (int j = 0; j < m; j++) {
31 |             if (matrix[i][j] == 'M') {
32 |                 for (auto & offset : offsets) {
33 |                     int oldI = i - offset[0];
34 |                     int oldJ = j - offset[1];
35 |
36 |                     if (oldI >= 0 && oldI < n && oldJ >= 0 && oldJ < m && matrix[oldI][oldJ] == 'M') {
37 |                         continue;
38 |                     }
39 |
40 |                     int len = 1;
41 |
42 |                     int newI = i + offset[0];
43 |                     int newJ = j + offset[1];
44 |
45 |                     while (newI >= 0 && newI < n && newJ >= 0 && newJ < m && matrix[newI][newJ] == 'M') {
46 |                         len++;
47 |                         newI += offset[0];
48 |                         newJ += offset[1];
49 |                     }
50 |
51 |                     ans = max(ans, len);
52 |                 }
53 |             }
54 |         }
55 |     }
56 |
57 |     cout << ans << endl;
58 |
59 |     return 0;
60 | }

```